

ANDROID DEVELOPER INTERNSHIP PROGRAM

Task 4

Task Title:

Design App Architecture (MVVM + Flow Diagram)

App Name: Eco-Cart

Proposed by: Ruhma Amir

1. MVVM Architecture Overview

Eco-Cart is designed using the **MVVM (Model-View-ViewModel)** architectural pattern. This ensures a "Separation of Concerns," making the app easy to test, maintain, and scale.

1. **Model:** Represents the data structure (e.g., Product class) and the business logic. It handles data from the Local JSON or API.
2. **View (UI Layer):** Built with **Flutter/Jetpack Compose**. It is "passive," meaning it only displays what the ViewModel tells it to. It observes the ViewModel for state changes.
3. **ViewModel:** The bridge between Data and UI. It holds the **State** (e.g., isScanning, productResult). It handles user actions (like clicking "Scan") and requests data from the Repository.

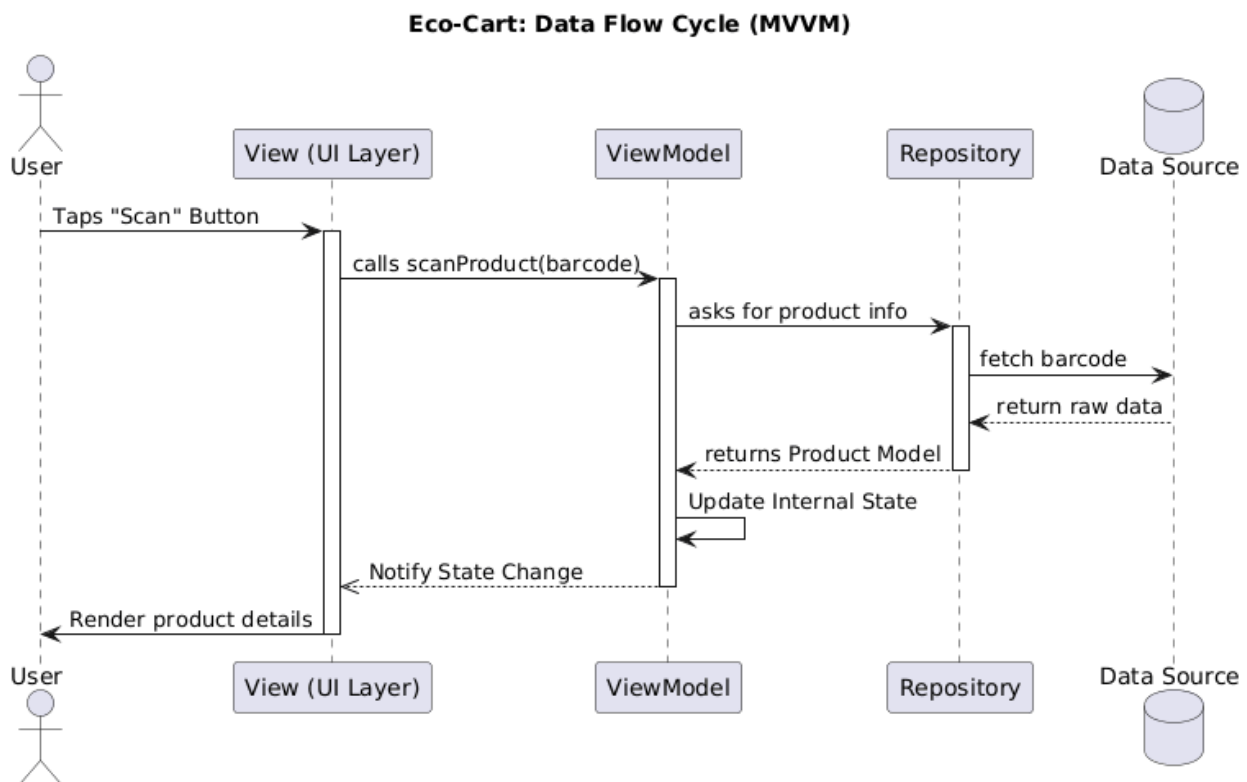
2. Defined App Layers

1. **UI Layer:** Contains Activities/Fragments or Widgets. It uses **StateFlow/LiveData** to reactively update the UI.
2. **Domain Layer (The Logic):** Contains "Use Cases" such as CalculateEcoScore or GetProductDetails. This is the brain of the app.

3. **Data Layer:** Includes the **Repository Pattern**. The Repository acts as a single point of truth, deciding whether to fetch data from the local assets (Mock JSON) or a future Remote API.

3. Data Flow (The Cycle)

1. **User Action:** User taps the "Scan" button on the **View**.
2. **Request:** The **View** calls a function in the **ViewModel**.
3. **Data Fetch:** The **ViewModel** asks the **Repository** for product info.
4. **Response:** The **Repository** returns the Product model to the **ViewModel**.
5. **State Update:** The **ViewModel** updates its internal state (e.g., showResult = true).
6. **UI Update:** The **View** (observing the state) automatically renders the product details on the screen.



4. Create MVVM Diagram

The Eco-Cart application follows the **MVVM (Model-View-ViewModel)** architectural pattern to ensure a strict separation of concerns, making the app modular and scalable.

1. UI Layer (The View)

Components: MainActivity, ProductDetailFragment, and ScannerActivity.

Responsibility: Handles user interactions (like tapping "Scan") and displays data. It is "lifecycle-aware" and remains passive, only reflecting the state provided by the ViewModel.

2. Domain Layer (The ViewModel)

Components: ProductViewModel, EcoScoreUseCase, and StateFlow.

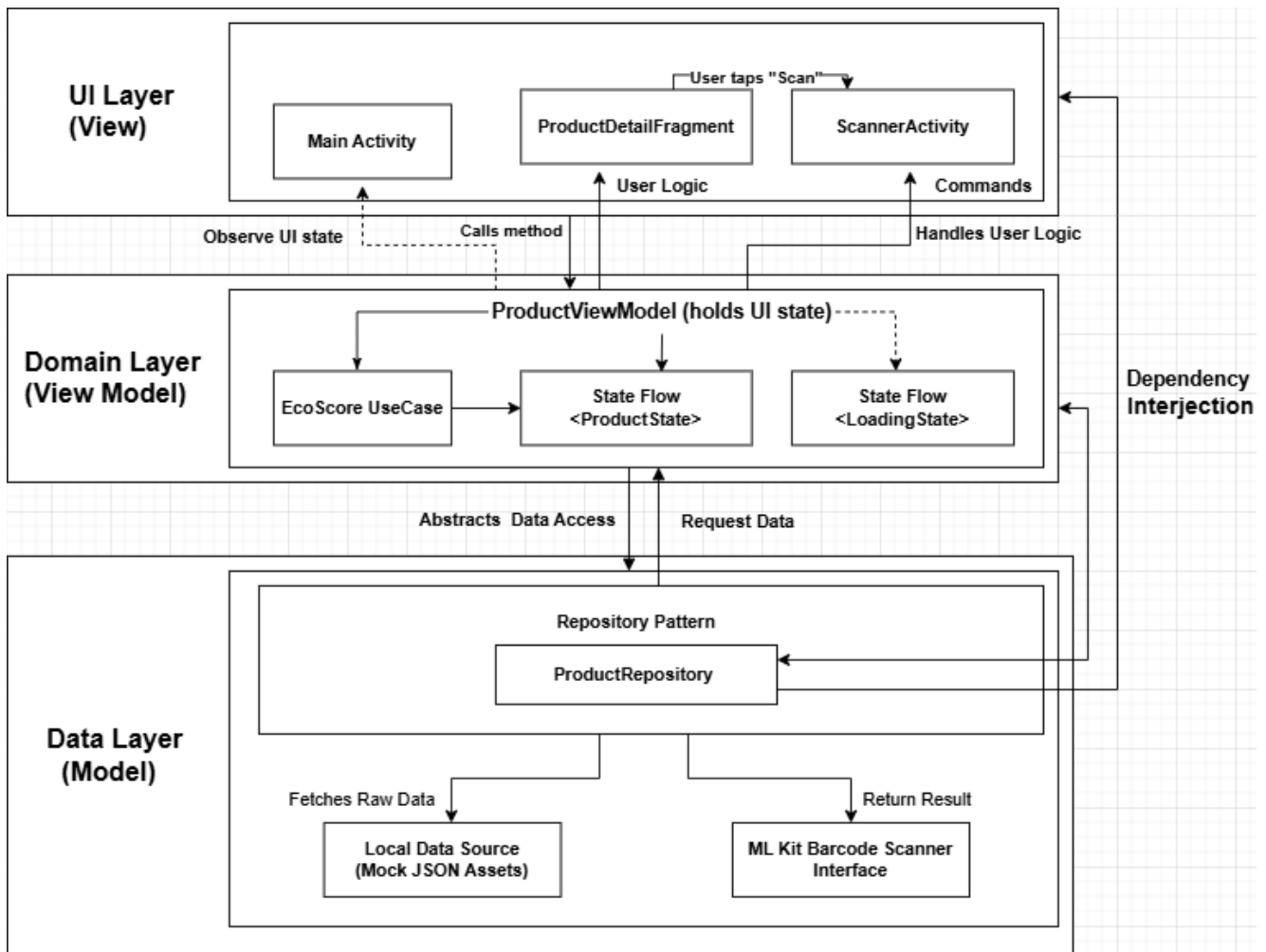
Responsibility: Acts as the bridge between the UI and Data layers. It maintains the **UI State** (e.g., loading, success, or error) using StateFlow. This prevents data loss during configuration changes (like screen rotation).

Logic: It triggers the EcoScoreUseCase to process business logic independently of the data source.

3. Data Layer (The Model)

Components: ProductRepository, Local Data Source, and ML Kit Interface.

Responsibility: Manages application data. The **Repository Pattern** is used to abstract the data source, meaning the ViewModel doesn't care if the data comes from a local JSON file or a remote API—the Repository handles that decision.



4. App flow diagram

ECO-CART APP FLOW

